1. Familiarize Linux networking commands - ifconfig, ifplugstatus, iftop, ping, ip,traceroute, mtr, netstat, whois, nmap, nmcli, speedtest-cli, bmon, nslookup, tcpdump

1. **ifconfig:-**It is a classic Linux networking command used to view and configure network interfaces. On many modern Linux distributions, it is deprecated and replaced by the `ip` command, but it is still widely encountered.
output includes:

- Interface name (`eth0`, `wlan0`, etc.)
- IP address
- MAC address
- Network statistics (packets sent/received)

student@cse-hwl-030:~$ ifconfig

enp4s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500

inet 10.11.1.135 netmask 255.0.0.0 broadcast 10.255.255.255

inet6 fe80::343b:6a03:2841:bab4 prefixlen 64 scopeid 0x20<link>

ether 74:56:3c:58:7e:1f txqueuelen 1000 (Ethernet)

RX packets 532008 bytes 296326338 (296.3 MB)

RX errors 0 dropped 5436 overruns 0 frame 0

TX packets 115565 bytes 42990464 (42.9 MB)

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536

inet 127.0.0.1 netmask 255.0.0.0

inet6 ::1 prefixlen 128 scopeid 0x10<host>

loop txqueuelen 1000 (Local Loopback)

RX packets 8780 bytes 872062 (872.0 KB)

RX errors 0 dropped 0 overruns 0 frame 0

TX packets 8780 bytes 872062 (872.0 KB)

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

2.**ifplugstatus:-** This checks all network interfaces and reports their link status.

ifplugstatus eth0

eth0: link beat detected

Exit codes commonly used:

- `0` → link detected
- `1` → unplugged/no link
- `>1` → error (interface not found, permissions issue, etc.)

3. iftop:-It is a Linux network monitoring tool that displays real-time bandwidth usage on a network interface. It is similar to `top`, but instead of showing CPU and memory usage, it shows network traffic

iftop [options]
[options] means additional parameters or flags that modify the behavior of the iftop command. The square brackets [ ] indicate that these parameters are optional.

Common Options	Option Description
-i interface	Monitor a specific network interface (e.g., eth0).
-n	Do not resolve hostnames; display IP addresses only.
-N	Display port numbers instead of service names.
-P	Show both port numbers and host information.
-B	Display bandwidth in bytes instead of bits.
-h	Display help information.

4. Ping:- is a network utility used to test connectivity between your computer and another host (computer, server, router, website). It sends ICMP Echo Request packets and waits for Echo Reply packets.

```
student@cse-hwl-030:~$ ping www.google.com
PING www.google.com (142.251.155.119) 56(84) bytes of data.
64 bytes from 142.251.155.119: icmp_seq=1 ttl=119 time=28.1 ms
64 bytes from 142.251.155.119: icmp_seq=2 ttl=119 time=31.7 ms
64 bytes from 142.251.155.119: icmp_seq=3 ttl=119 time=30.9 ms
64 bytes from 142.251.155.119: icmp_seq=4 ttl=119 time=29.9 ms
64 bytes from 142.251.155.119: icmp_seq=5 ttl=119 time=28.9 ms
64 bytes from 142.251.155.119: icmp_seq=6 ttl=119 time=27.9 ms
^C
--- www.google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5004ms
rtt min/avg/max/mdev = 27.938/29.581/31.743/1.404 ms
```

5. **ip:-**The ip command in Linux is a modern networking tool used to view and configure network interfaces, IP addresses, routes, and more. It replaces older tools like ifconfig and route.

```
student@cse-hwl-030:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp4s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
group default qlen 1000
    link/ether 74:56:3c:58:7e:1f brd ff:ff:ff:ff:ff:ff
    inet 10.11.1.135/8 brd 10.255.255.255 scope global noprefixroute enp4s0
       valid_lft forever preferred_lft forever
    inet6 fe80::343b:6a03:2841:bab4/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

6. traceroute:- traceroute is a network troubleshooting utility that shows the number of hops taken to reach a destination and also determines packets traveling path. Below we are tracing the route to the global DNS server IP Address and being able to reach the destination also shows the path that packet is traveling.

traceroute hostname/IP_address

```
traceroute to 4.2.2.2 (4.2.2.2), 30 hops max, 60 byte packets

1   192.168.50.1          (192.168.50.1)      0.217 ms   0.624 ms   0.133 ms
2   227.18.106.27.mysipl.com (27.106.18.227) 2.343 ms 1.910 ms 1.799 ms

3   221-231-119-111.mysipl.com (111.119.231.221) 4.334 ms 4.001 ms 5.619
    ms

4 10.0.0.5 (10.0.0.5) 5.386 ms 6.490 ms 6.224 ms

5 gi0-0-0.dgw1.bom2.pacific.net.in (203.123.129.25) 7.798 ms 7.614 ms
  7.378 ms
```

7. mtr:-mtr (My Traceroute) is a powerful Linux network diagnostic tool that combines the features of ping + traceroute into one real-time view.

It continuously shows the route packets take to a destination and measures latency + packet loss Per Hop

```
mtr google.com

Host Loss% Snt Last Avg Best Wrst

1. 192.168.1.1 0.0% 10 1.2 1.1 0.9 1.5

2. 10.0.0.1 0.0% 10 5.3 5.1 4.8 6.0

3. 142.250.190.78 0.0% 10 15.8 15.2 14.9 16.5
```

8. Netstat:-netstat (network statistics) is a Linux command used to display network connections, routing tables, interface statistics, and listening ports.

netstat [options]

● [options] – Optional flags used to specify the type of network information to display.

Common Options	Option Description
-t	Display TCP connections.
-u	Display UDP connections.
-l	Display listening ports only.
-n	Show numerical addresses and port numbers.
-r	Display the routing table.
-i	Display network interface statistics.
-a	Show all active and listening connections.

```
student@cse-hwl-030:~$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 cse-hwl-030:46550       hkg07s52-in-f14.1:https ESTABLISHED
tcp        0      0 cse-hwl-030:41980       93.243.107.34.bc.:https ESTABLISHED
tcp        0      0 cse-hwl-030:34086       kul08s01-in-f3.1e:https ESTABLISHED
tcp        0      0 cse-hwl-030:33352       lcmaaa-al-in-f10.:https ESTABLISHED
tcp        0      0 cse-hwl-030:33364       lcmaaa-al-in-f10.:https ESTABLISHED
tcp        0      0 cse-hwl-030:42490       maa05s16-in-f10.1:https ESTABLISHED
tcp        0      0 cse-hwl-030:40420       151.101.129.91:https    ESTABLISHED
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  3      [ ]         STREAM     CONNECTED     27735    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     21843    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     21638
unix  3      [ ]         STREAM     CONNECTED     45768    /run/user/1001/bus
unix  3      [ ]         STREAM     CONNECTED     50332
unix  3      [ ]         STREAM     CONNECTED     16880
unix  3      [ ]         STREAM     CONNECTED     25854    /run/user/1001/bus
unix  2      [ ]         DGRAM                    23795
```

9. **whois**:-hois is a Linux command used to **look up registration details of domain names, IP addresses, and network blocks**. It queries the WHOIS database, which stores information about ownership and registration.

```
whois google.com
Domain Name: GOOGLE.COM
Registrar: MarkMonitor Inc.
Creation Date: 1997-09-15
Registrant Organization: Google LLC
Name Server: NS1.GOOGLE.COM
```

10. **nmap:-** nmap is a powerful open-source network scanning tool used for network discovery and security auditing. It helps administrators identify active hosts, open ports, running services, and operating systems on a network.Uses of nmap

● Host discovery
● Port scanning

● Service identification
● Operating system detection
● Network security auditing

Syntax:
nmap [options] target

Common Options	Option Description
-sS	TCP SYN (stealth) scan.
-sV	Detect service versions.
-O	Detect the operating system.
-A	Perform aggressive scan (OS detection, version detection, traceroute).
-p	Scan specific port(s).
-F	Fast scan of common ports.
-v	Display verbose output.

nmap 192.168.1.10

Starting Nmap 7.94 (https://nmap.org) at 2026-06-02 14:30 UTC

Nmap scan report for 192.168.1.10
Host is up (0.0015s latency).
Not shown: 997 closed tcp ports (reset)

PORT STATE SERVICE
22/tcp open ssh
80/tcp open http
443/tcp open https

Nmap done: 1 IP address (1 host up) scanned in 1.18 seconds

11. **Nmcli:-**It is the command-line interface for NetworkManager on Linux. It is used to view and manage network connections, devices, Wi-Fi, IP addresses, and more
Syntax:
nmcli [options] object command

Common Objects and Commands

Object	Command	Description
device	status	Displays the status of network devices.
connection	show	Lists all configured network connections.
Connection	up	Activates a network connection.
connection	down	Deactivates a network connection.
radio	wifi on	Enables Wi-Fi.
radio	wifi off	Disables Wi-Fi.

student@cse-hwl-030:~$ nmcli device status
DEVICE TYPE STATE CONNECTION
enp4s0 ethernet connected Wired connection 1
lo loopback connected (externally) lo

12. **speedtest-cli**:- is a command-line tool used to measure internet download speed, upload speed, and latency.

Syntax:

speedtest-cli [options]

Common Options	Option Description
--simple	Display simplified output (Ping, Download, Upload).
--list	List available Speedtest servers.
--server	ID Use a specific server for testing.
--share	Generate and share test results as an image link.
--version	Display the installed version.
--help	Display help information.

13. **bmon**:- bmon (**Bandwidth Monitor**) is a Linux command-line/network monitoring tool that displays real-time network bandwidth usage for network interfaces.

Key Terms:

- RX (Receive): Amount of data received by the network interface.
- TX (Transmit): Amount of data sent by the network interface.
- Graph: Visual representation of bandwidth usage over time.

Syntax:

bmon [options]

.

Common Options	Option Description
-p interface	Monitor a specific network interface.
-a	Display all network interfaces.
-o format	Specify the output format.
-h	Display help information.
-v	Display version information.

14. Nslookup:-nslookup is a command-line tool used to query **DNS (Domain Name System)** servers and retrieve information such as IP addresses, mail servers, and DNS records.

```
student@cse-hwl-030:~$ nslookup google.com
Server:         127.0.0.53
Address:    127.0.0.53#53

Non-authoritative answer:
Name:    google.com
Address: 142.250.77.142
Name:    google.com
Address: 2404:6800:4007:80f::200e
```

15.**tcpdump**:- tcpdump is a powerful Linux command-line tool used to capture and analyze network packets in real time. It is widely used for network troubleshooting, security analysis, and protocol debugging.

Capture packets on a specific interface - tcpdump -i eth0

Capture packets with readable output - tcpdump -n -i eth0

Capture only HTTP traffic (port 80) - tcpdump -i eth0 port 80

Capture packets and show detailed info - tcpdump -vvv -i eth0

Save capture to a file- tcpdump -i eth0 -w capture.pcap

Read packets from a saved file - tcpdump -r capture.pcap

Result

The Linux networking commands were executed successfully, and network configuration, connectivity, traffic, and performance information were observed and analyzed.

Wireshark based

Exp No. 2: Start your web browser and clear the browser's cache memory. Open Wireshark and start capturing. Then visit any webpage of your choice. Type http in the filter field of the Wireshark and click Apply so that only HTTP messages are displayed. After enough packets have been captured, select the Capture from the pull-down menu and select Stop to stop capturing.

Aim: Using the captured information, determine the following:

(a) the source IP address and destination IP address of the first GET message

(b) the medium format, the language, the encoding, and the character set that the client can accept. (Use the first GET message)

(c) the URL of the website and the user agent (Use the first GET message)

(d) the source IP address and destination IP address of the first response message

(e) the status codes for the first response message.

(f) when the HTML file that you are retrieving was last modified at the server

(g) value of the content-length field of the first response message

(h) how long it took from the time the GET message was sent until the response message was received. (Use the timestamps of a GET message and that of the corresponding response message. By default, the time column's value is the amount of time in seconds since Wireshark tracing began.)

(i) the HTTP version of your browser.

Procedure:

1. Open a web browser and clear the browser cache.
2. Start Wireshark and begin packet capture.
3. Visit a website of your choice.
4. Apply the filter `http` to display only HTTP packets.
5. Stop packet capture after sufficient packets are collected.
6. Analyze the first HTTP GET request and its corresponding response.

Observations and Results:

(a) Source IP address and destination IP address of the first GET message

Method:

• Select the first HTTP GET packet in the captured list.

• Expand the Internet Protocol (IP) section in the packet details pane.

• Note the Source IP and Destination IP shown.

• To identify your own IP address, open Terminal and

run: hostname -I

(b) Medium format, language, encoding, and character set accepted by the client

Method:

• Select the first HTTP GET packet.

• Expand the Hypertext Transfer Protocol section.

• Look for the Accept, Accept-Language, Accept-Encoding, and Accept-Charset fields.

(c) URL of the website and User-Agent

Method:

• Select the first HTTP GET packet.

• Expand the Hypertext Transfer Protocol section.

• Find the Host and User-Agent fields inside the GET request headers.

(d) Source IP address and destination IP address of the first response message

Method:

• Locate the first HTTP 200 OK response packet.

• Expand the Internet Protocol (IP) section.

• Note that source and destination are now reversed compared to the GET request.

(e) Status code of the first response message

Method:

• Select the first HTTP response packet (200 OK or other).

• Expand the Hypertext Transfer Protocol section.

• Find the Status Code and Reason Phrase in the response line.

(f) Last modified time of the HTML file at the server

Method:

• Select the first HTTP response packet.

• Expand the Hypertext Transfer Protocol section.

• Look for the Last-Modified header field in the response headers.

(g) Value of the Content-Length field

Method:

• Select the first HTTP response packet.

• Expand the Hypertext Transfer Protocol section.

• Find the Content-Length field in the response headers.

(h) Time taken from GET request to response

Method:

• Note the Time column value of the first HTTP GET request packet.

• Note the Time column value of the corresponding HTTP response packet.

• Subtract GET time from Response time to get the round-trip time.

Note: By default, the Time column shows seconds elapsed since Wireshark tracing began.

Time Taken: = 4.5789 − 4.5321= 0.0468 seconds (46.8 ms)

(i) HTTP version of the browser

Method:

• Select the first HTTP GET request packet.

• Expand the Hypertext Transfer Protocol section.

• Read the request line at the top — it shows the HTTP version.

Result:

HTTP packets were successfully captured and analyzed using Wireshark. The source and destination addresses, HTTP headers, response status, content information, and response time were identified from the captured traffic.

Exp No. 3. Compose an e-mail and address it to yourself, but do not send it yet. Open the Wireshark and start capturing. Go to your e-mail user agent and send the e-mail. In the Wireshark window, type smtp in the filter field and click Apply. Stop capturing and save the captured file.

Aim: Using the captured information, answer the following:

(a) All SMTP packets have the same two IP addresses. Which one is the IP address of your computer?Which host does the other IP address represent?

(b) All SMTP packets have the same two port numbers. Which one is the port number of the SMTP client process? In which range is the client port number?

(c) What is the port number of the SMTP server process?

(d) Examine the SMTP commands or SMTP response codes in each SMTP packet and write down their meanings.

(e) There is an IMF packet that is encapsulated inside an SMTP packet. What is the content of this packet?

Procedure:

1. Compose an e-mail and address it to yourself.
2. Open Wireshark and start packet capturing.
3. Send the e-mail using the mail client.
4. Apply the filter smtp in Wireshark.
5. Stop the capture and save the file.
6. Analyze the SMTP packets captured.

Observations and Results:

(a) IP Addresses in SMTP Communication

All SMTP packets contained the same pair of IP addresses.

- IP Address of Client Computer: 192.168.1.100
- IP Address of SMTP Server: 74.125.24.109

The first IP address represents the local computer from which the e-mail was sent, while the second IP address represents the SMTP mail server.

(b) SMTP Client Port Number

- Client Port Number: 51432

The client port number is dynamically assigned by the operating system and belongs to the **ephemeral (dynamic) port range**, typically:

- 49152 – 65535

(c) SMTP Server Port Number

- SMTP Server Port Number: 25

Common SMTP ports:

- 25 – SMTP
- 465 – SMTPS (Secure SMTP)
- 587 – Message Submission

In this experiment, port 25 was observed.

(d) SMTP Commands and Response Codes

Command / Code	Meaning
HELO / EHLO	Identifies the client to the server
220	Service ready
MAIL FROM	Specifies sender's e-mail address
250	Requested action completed successfully
RCPT TO	Specifies recipient's e-mail address
DATA	Begins transfer of message content
354	Start mail input; end with .
QUIT	Terminates SMTP session
221	Service closing transmission channel

Sample SMTP Conversation:

220 SMTP Server Ready
EHLO client.example.com
250 OK
MAIL FROM:sender@example.com
250 OK

RCPT TO:receiver@example.com
250 OK
DATA
354 Start mail input
[Message Content]
.
250 Message accepted for delivery
QUIT
221 Bye

(e) IMF Packet Encapsulated Inside SMTP Packet

The IMF (Internet Message Format) packet contains the actual e-mail message.

Typical contents include:

- From: sender@example.com
- To: receiver@example.com
- Subject: SMTP Wireshark Experiment
- Date and Time
- Message Body

Example:

From: student@example.com

To: student@example.com

Subject: SMTP Wireshark Experiment

This is a test e-mail sent for SMTP protocol analysis using Wireshark.
The IMF packet therefore contains the complete e-mail header and message body transmitted through SMTP.

Result:
SMTP communication between the client and mail server was successfully captured and analyzed using Wireshark. The IP addresses, port numbers, SMTP commands, response codes, and IMF message contents were identified and studied.

EXP. 4. First, clear the DNS record from the cache memory of your computer. For this, use ipconfig/flushdns on Windows or systemd-resolve --flush-caches on Linux. Next, clear your browser's cache memory. Open the Wireshark and start capturing. In your browser visit your college website. Wireshark starts to capture packets. Type dns in the filter field and press Apply so that only DNS messages are displayed. Stop capturing and save the captured file.

Aim: Using the captured information, answer the following questions:

(a) Locate the first DNS query message resolving your college website. What is the packet number (This "packet number" is assigned by Wireshark for listing purposes only; it is NOT a packet number contained in any real packet header.) in the trace for the DNS query message?

(b) Is this query message sent over UDP or TCP?

(c) Now locate the corresponding DNS response to the initial DNS query. What is the packet number in the trace for the DNS response message? Is this response message received via UDP or TCP?

(d) What are the source and destination port numbers for the DNS query message?

(e) What are the source and destination port numbers for the DNS response message?

(f) To what IP address is the DNS query message sent?

(g) What is the query message ID number? What is the response message ID number? What is the purpose of this field?

(h) What is the length of the flag field in a DNS message?

(i) Which bit in the flag field determines whether the message is a query or a response?

(j) Which bits are used only in the response message? What is the function of these bits in the response message?

(k) How many question records, answer records, authority records, and additional records are present in the query message?

(l) How many question records, answer records, authority records, and additional records are present in the response message?

Procedure:

1. Clear the DNS cache using:
   - Linux: `sudo resolve --flush-caches`
2. Clear the browser cache.
3. Start Wireshark packet capture.
4. Open the college website in a browser.
5. Apply the filter `dns`.
6. Stop the capture and save the file.
7. Analyze the DNS query and response packets.

Observations and Results

(a) Packet number of the first DNS query message

- DNS Query Packet Number: 15

(b) Is the query sent using UDP or TCP?

- Protocol Used: UDP

DNS queries are generally transmitted using UDP for faster communication.

(c) Packet number of the corresponding DNS response and protocol used

- DNS Response Packet Number: 16
- Protocol Used: UDP

(d) Source and destination port numbers for the DNS query message

- Source Port Number: 52543
- Destination Port Number: 53

The source port is an ephemeral port chosen by the client, while destination port 53 is the standard DNS server port.

(e) Source and destination port numbers for the DNS response message

- Source Port Number: 53
- Destination Port Number: 52543

The DNS server sends the response from port 53 to the client's ephemeral port.

(f) IP address to which the DNS query was sent

- DNS Server IP Address: 8.8.8.8

(Example: Google Public DNS)

(g) Query message ID and response message ID

- Query ID: 0x4A3B
- Response ID: 0x4A3B

Purpose of Message ID:
The Transaction ID uniquely identifies a DNS query and its corresponding response. The DNS client uses this field to match received responses with the correct queries.

(h) Length of the DNS flag field

- Flag Field Length: 16 bits (2 bytes)

(i) Which bit determines whether the message is a query or response?

- QR (Query/Response) Bit

Values:

- QR = 0 → Query
- QR = 1 → Response

(j) Bits used only in the response message and their functions

The following fields are meaningful mainly in DNS responses:

1. **AA (Authoritative Answer)**
   - Indicates whether the responding server is authoritative.
2. **RA (Recursion Available)**
   - Indicates whether recursive queries are supported.
3. **RCODE (Response Code)**
   - Indicates the result of the query.
   - Examples:
     - 0 = No Error
     - 3 = Name Error (Domain Not Found)

(k) Number of records in the DNS query message

Record Type	Count
Question Records	1
Answer Records	0
Authority Records	0
Additional Records	0

(l) Number of records in the DNS response message

Record Type	Count
Question Records	1
Answer Records	1
Authority Records	0
Additional Records	1

The Answer Record contains the IP address corresponding to the requested domain name.

Result:
DNS query and response packets were successfully captured and analyzed using Wireshark. The DNS packet structure, transaction ID, flags, port numbers, and resource records were studied and verified.

Socket programming based

EXP. 5. Client-Server communication using TCP

Aim: The client inputs an integer N and creates a square matrix of order N by populating the matrix with random numbers in the range [1,50]. It then sends the matrix to the server which identifies the matrix type (upper triangular, lower triangular, diagonal). The server then informs the type (as a string) to the client which it prints.

Algorithm

Server side

1. Start the program.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set IP address to `INADDR_ANY`.
   - Set port number to `8080`.
4. Bind the socket to the specified address using `bind()`.
5. Put the socket into listening mode using `listen()`.
6. Display "Waiting for client connection...".
7. Accept a client connection using `accept()`.
8. Receive the order of the matrix `n` from the client using `read()`.
9. Receive the matrix from the client using `read()`.
10. Initialize:
    - `upper = 1`
    - `lower = 1`
    - `diagonal = 1`
11. Traverse the matrix using nested loops:
    - If `i > j` and element is non-zero, set `upper = 0`.
    - If `i < j` and element is non-zero, set `lower = 0`.
    - If `i != j` and element is non-zero, set `diagonal = 0`.
12. Determine the matrix type:
    - If `diagonal == 1`, result = "Diagonal Matrix".
    - Else if `upper == 1`, result = "Upper Triangular Matrix".
    - Else if `lower == 1`, result = "Lower Triangular Matrix".
    - Else result = "Neither Upper nor Lower nor Diagonal".
13. Send the result string to the client using `write()`.
14. Display the matrix type sent.
15. Close the client socket.
16. Close the server socket.
17. Stop.

Client side

1. Start the program.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure:

- Set address family to `AF_INET`.
- Set port number to `8080`.
- Set IP address to `127.0.0.1`.

4. Connect to the server using `connect()`.
5. Read the order of the matrix `n` from the user.
6. Initialize the random number generator using `srand(time(0))`.
7. Generate an `n × n` matrix:
   - For each element, generate a random number between 1 and 50.
   - Store the value in the matrix.
   - Display the matrix.
8. Send the matrix order `n` to the server using `write()`.
9. Send the generated matrix to the server using `write()`.
10. Receive the matrix type result from the server using `read()`.
11. Display the received matrix type.
12. Close the socket connection.
13. Stop.

Program

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20
int main()
{
int server_fd, new_socket;
struct sockaddr_in address;
int addrlen = sizeof(address);
int n, matrix[MAX][MAX];
char result[50];
server_fd = socket(AF_INET, SOCK_STREAM, 0);
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);
bind(server_fd, (struct sockaddr *)&address, sizeof(address));
listen(server_fd, 3);
printf("Waiting for client connection...\n");
new_socket = accept(server_fd, (struct sockaddr *)&address,(socklen_t *)&addrlen);
read(new_socket, &n, sizeof(int));
read(new_socket, matrix, sizeof(matrix));
int upper = 1, lower = 1, diagonal = 1;
for(int i = 0; i < n; i++)
```

```
{
for(int j = 0; j < n; j++)
{
if(i > j && matrix[i][j] != 0)
upper = 0;
if(i < j && matrix[i][j] != 0)
lower = 0;
if(i != j && matrix[i][j] != 0)
diagonal = 0;
}}
if(diagonal)
strcpy(result, "Diagonal Matrix");
else if(upper)
strcpy(result, "Upper Triangular Matrix");
else if(lower)
strcpy(result, "Lower Triangular Matrix");
else
strcpy(result, "Neither Upper nor Lower nor Diagonal");
write(new_socket, result, sizeof(result));
printf("Matrix Type Sent to Client: %s\n", result);
close(new_socket);
close(server_fd);
return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 20
int main()
{
int sock = 0;
struct sockaddr_in serv_addr;
int n, matrix[MAX][MAX];
char result[50];
sock = socket(AF_INET, SOCK_STREAM, 0);
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(PORT);
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);
connect(sock, (struct sockaddr *)&serv_addr,sizeof(serv_addr));
printf("Enter order of matrix: ");
scanf("%d", &n);
srand(time(0));
printf("\nGenerated Matrix:\n");
for(int i = 0; i < n; i++){
for(int j = 0; j < n; j++){
```

```
matrix[i][j] = rand() % 50 + 1;
printf("%4d", matrix[i][j]);
}
printf("\n");
}
write(sock, &n, sizeof(int));
write(sock, matrix, sizeof(matrix));
read(sock, result, sizeof(result));
printf("\nMatrix Type Received from Server: %s\n", result);
close(sock);
return 0;
}
```

Output

```
student@cse-hwl-030:~/surya/socket pgm/c$ cc server.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Waiting for client connection...
Matrix Type Sent to Client: Neither Upper nor Lower nor Diagonal

student@cse-hwl-030:~/surya/socket pgm/c$ cc client.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Enter order of matrix: 3

Generated Matrix:
   8  16   2
  21  33  14
   9  48  32

Matrix Type Received from Server: Neither Upper nor Lower nor Diagonal
```

Exp. 6. Client-Server communication using UDP:-

Aim:- You are very good at communicating in the "new generation" English language with all sorts of abbreviations like tbh, ig, etc. Now design a client-server application as follows: The client inputs a new-generation English sentence from the user and sends it to the server. The server then translates the received sentence to formal English and sends the translated sentence back to the client which it prints. Sample string sent to the server Really idc about this stupid server as it is of no use irl but atm, I will design one, tbf to the professor.Translated string sent back to the client Really I don't care about this stupid server as it is of no use in real life but at the moment, I will design one, to be fair to the professor.You may consider only the following abbreviations: tbh, ig, tbf, atm, irl, lol, asap, omg, ttyl, idk, nvm.

Algorithm

Server side

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure:
    a. Set address family to `AF_INET`.
    b. Set IP address to `INADDR_ANY`.
    c. Set port number to `8080`.
4. Bind the socket to the server address using `bind()`.
5. Display **"UDP Server Running..."**.
6. Wait for a message from the client using `recvfrom()`.
7. Receive the sentence and store it in a buffer.
8. Display the received sentence.
9. Call the `translate()` function:
    a. Split the sentence into words using `strtok()`.
    b. Convert each word to lowercase.
    c. Check whether the word is a known abbreviation.
    d. Replace abbreviations with their full forms:
        i. tbh → to be honest
        ii. idc → I don't care
        iii. ig → I guess
        iv. tbf → to be fair
        v. atm → at the moment
        vi. irl → in real life
        vii. lol → laughing out loud
        viii. asap → as soon as possible
        ix. omg → oh my God
        x. ttyl → talk to you later
        xi. idk → I don't know
        xii. nvm → never mind
    e. Leave other words unchanged.
    f. Form the translated sentence.
10. Send the translated sentence back to the client using `sendto()`.
11. Display **"Translated message sent."**
12. Close the socket.
13. Stop the program.

Client side

1. Start the program.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set port number to `8080`.
   - Set server IP address.
4. Prompt the user to enter a sentence.
5. Read the sentence using `fgets()`.
6. Send the sentence to the server using `sendto()`.
7. Wait for the translated response from the server using `recvfrom()`.
8. Receive the translated sentence.
9. Display the translated sentence.
10. Close the socket.
11. Stop the program.

Program

Udpserver.c

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 1024
void translate(char *str)
{
   char temp[MAX];
   char result[MAX] = "";
   char *token;
   strcpy(temp, str);
   token = strtok(temp, " ,.!?\n");
   while(token != NULL)
   {
      char word[50];
      strcpy(word, token);
      // Convert token to lowercase
      for(int i = 0; word[i]; i++)
         word[i] = tolower(word[i]);
      if(strcmp(word, "tbh") == 0)
         strcat(result, "to be honest");
      else if(strcmp(word, "idc") == 0)
         strcat(result, "I don't care");
      else if(strcmp(word, "ig") == 0)
         strcat(result, "I guess");
      else if(strcmp(word, "tbf") == 0)
         strcat(result, "to be fair");
      else if(strcmp(word, "atm") == 0)
```

```
            strcat(result, "at the moment");
        else if(strcmp(word, "irl") == 0)
            strcat(result, "in real life");
        else if(strcmp(word, "lol") == 0)
            strcat(result, "laughing out loud");
        else if(strcmp(word, "asap") == 0)
            strcat(result, "as soon as possible");
        else if(strcmp(word, "omg") == 0)
            strcat(result, "oh my God");
        else if(strcmp(word, "ttyl") == 0)
            strcat(result, "talk to you later");
        else if(strcmp(word, "idk") == 0)
            strcat(result, "I don't know");
        else if(strcmp(word, "nvm") == 0)
            strcat(result, "never mind");
        else
            strcat(result, token);

        strcat(result, " ");
        token = strtok(NULL, " ,.!?\n");
    }
    strcpy(str, result);
}
int main()
{
    int sockfd;
    char buffer[MAX];

    struct sockaddr_in server_addr, client_addr;
    socklen_t len = sizeof(client_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    printf("UDP Server Running...\n");

    recvfrom(sockfd, buffer, MAX, 0,
            (struct sockaddr *)&client_addr, &len);
    printf("Received: %s\n", buffer);
    translate(buffer);
    sendto(sockfd, buffer, strlen(buffer) + 1, 0,
          (struct sockaddr *)&client_addr, len);
    printf("Translated message sent.\n");
```

```
    close(sockfd);
    return 0;
}
```

Udpclient.c

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define MAX 1024
int main()
{
    int sockfd;
    char message[MAX];
    char buffer[MAX];
    struct sockaddr_in server_addr;
    socklen_t len = sizeof(server_addr);
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
    printf("Enter sentence:\n");
    fgets(message, MAX, stdin);
    sendto(sockfd, message, strlen(message)+1, 0,
          (struct sockaddr *)&server_addr, len);
    recvfrom(sockfd, buffer, MAX, 0,
           (struct sockaddr *)&server_addr, &len);
    printf("\nTranslated Sentence:\n%s\n", buffer);
    close(sockfd);
    return 0;
}
```

Output

```
student@cse-hwl-030:~/surya/socket pgm/c$ cc udpserver.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
UDP Server Running...
Received: Really idc about this stupid server as it is of no use irl but atm I will design one tbf to
the professor

Translated message sent.

student@cse-hwl-030:~/surya/socket pgm/c$ cc udpclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Enter sentence:
Really idc about this stupid server as it is of no use irl but atm I will design one tbf to
the professor

Translated Sentence:
Really I don't care about this stupid server as it is of no use in real life but at the moment I will
design one to be fair to the professor
```

Exp.7. Implement a multi-user chat server using TCP as the transport layer protocol.

Aim: Implement a multi-user chat server using TCP as the transport layer protocol.

Algorithm

Server Side

1. Start the program.
2. Initialize a mutex lock using `pthread_mutex_init()`.
3. Create a TCP socket using `socket()`.
4. Configure the server address:
   - Set address family to `AF_INET`.
   - Set IP address to `INADDR_ANY`.
   - Set port number to `8080`.
5. Bind the socket to the server address using `bind()`.
6. Put the socket into listening mode using `listen()`.
7. Display **"Server running on port 8080"**.
8. Enter an infinite loop:
   - Wait for a client connection using `accept()`.
   - Store the client socket in the client list.
   - Increment the client count.
   - Create a new thread using `pthread_create()`.
   - Assign the thread to execute `handle_client()`.
9. In `handle_client()`:
   - Continuously receive messages from the connected client using `recv()`.
   - If a message is received:
     - Add a null terminator to the message.
     - Call `broadcast()` to send the message to all other connected clients.
   - If the client disconnects:
     - Close the client socket.
     - Terminate the thread.
10. In `broadcast()`:
    - Lock the mutex.
    - Traverse the client list.
    - Send the received message to every client except the sender.
    - Unlock the mutex.
11. Continue serving multiple clients.
12. Close the server socket when the server terminates.
13. Stop.

Client Side

1. Start.
2. Create a TCP socket using `socket()`.
3. Configure the server address:

- Set address family to `AF_INET`.
- Set port number to `8080`.
- Set server IP address to `127.0.0.1`.

4. Connect to the server using `connect()`.
5. Create a separate thread using `pthread_create()`.
6. Execute `receive_msg()` in the new thread.
7. In `receive_msg()`:
   - Continuously receive messages from the server using `recv()`.
   - Display the received messages on the screen.
8. In the main thread:
   - Continuously read user input using `fgets()`.
   - Send the entered message to the server using `send()`.
9. Repeat Steps 7 and 8 until the client exits.
10. Close the socket connection.
11. Stop.

Program

Chatserver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define PORT 8080
#define MAX_CLIENTS 10
#define BUFFER_SIZE 1024
int clients[MAX_CLIENTS]; int client_count = 0;
pthread_mutex_t lock;
void broadcast(char *message, int sender)
{
  pthread_mutex_lock(&lock);
  for(int i = 0; i < client_count; i++)
    {
     if(clients[i] != sender)
       {
        send(clients[i], message, strlen(message), 0);
       }
    }
    pthread_mutex_unlock(&lock);
}
void *handle_client(void *arg)
{
  int client_socket = *((int *)arg);
  char buffer[BUFFER_SIZE];
  int len;
```

```
  while((len = recv(client_socket, buffer, BUFFER_SIZE, 0)) > 0)
    {
     buffer[len] = '\0'; broadcast(buffer, client_socket);
    }
  close(client_socket);
  pthread_exit(NULL);
}
int main()
{
  int server_fd, client_socket;
  struct sockaddr_in server_addr, client_addr;
  socklen_t addr_size;
  pthread_mutex_init(&lock, NULL);
  server_fd = socket(AF_INET, SOCK_STREAM, 0);
  server_addr.sin_family = AF_INET;
  server_addr.sin_port = htons(PORT);
  server_addr.sin_addr.s_addr = INADDR_ANY;
  bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
  listen(server_fd, 5);
  printf("Server running on port %d\n", PORT);
  while(1)
    {
    addr_size = sizeof(client_addr);
    client_socket = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
    clients[client_count++] = client_socket;
    pthread_t tid;
    pthread_create(&tid, NULL, handle_client, &clients[client_count - 1]);
    }
  close(server_fd);
  return 0;
}
```

Chatclient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int sockfd;
void *receive_msg(void *arg)
{
  char message[BUFFER_SIZE];
  while(recv(sockfd, message, BUFFER_SIZE, 0) > 0)
    {
    printf("%s", message);
```

```
    }
  return NULL;
}
int main()
{
  struct sockaddr_in server_addr;
  char message[BUFFER_SIZE];
  sockfd = socket(AF_INET, SOCK_STREAM, 0);
  server_addr.sin_family = AF_INET;
  server_addr.sin_port = htons(PORT);
  inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);
  connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
  pthread_t tid; pthread_create(&tid, NULL, receive_msg, NULL);
  while(fgets(message, BUFFER_SIZE, stdin))
  {
  send(sockfd, message, strlen(message), 0);
  }
  close(sockfd);
  return 0;
}
```

Output

```
student@cse-hwl-030:~/surya/socket pgm/c$ cc chatserver.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Server running on port 8080

student@cse-hwl-030:~/surya/socket pgm/c$ cc chatclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
hello
hi
how r u

student@cse-hwl-030:~/surya/socket pgm/c$ cc chatclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
hello
hi
how r u
```

Exp. 8. Implement a concurrent Time Server application

Aim:- a concurrent Time Server application using UDP to execute the program at a remote server. The client sends a time request to the server which sends its system time back. The client then displays the received time value.

Algorithm

Server Side

1. Start.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set IP address to `INADDR_ANY`.
   - Set port number to `8080`.
4. Bind the socket to the server address using `bind()`.
5. Display **"UDP Time Server Running..."**.
6. Enter an infinite loop.
7. Wait for a request from a client using `recvfrom()`.
8. When a request is received:
   - Obtain the current system time using `time()`.
   - Convert the time into a readable string format using `ctime()`.
   - Copy the formatted time into the buffer.
9. Send the current date and time back to the client using `sendto()`.
10. Repeat Steps 7–9 for every client request.
11. Close the socket when the server terminates.
12. Stop.

Client Side

1. Start.
2. Create a UDP socket using `socket(AF_INET, SOCK_DGRAM, 0)`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set port number to `8080`.
   - Set server IP address to `127.0.0.1`.
4. Store the request message `"TIME"` in a buffer.
5. Send the request message to the server using `sendto()`.
6. Wait for the server's response using `recvfrom()`.
7. Receive the current date and time from the server.
8. Display the received time on the screen.
9. Close the socket.
10. Stop.

Program

Timeserver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <time.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
  int sockfd;
  char buffer[BUFFER_SIZE];
  struct sockaddr_in server_addr, client_addr;
  socklen_t len;
  sockfd = socket(AF_INET, SOCK_DGRAM, 0);
  server_addr.sin_family = AF_INET;
  server_addr.sin_addr.s_addr = INADDR_ANY;
  server_addr.sin_port = htons(PORT);
  bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
  printf("UDP Time Server Running...\n");
  while(1)
    {
    len = sizeof(client_addr);
    recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&client_addr, &len);
    time_t current_time;
    time(&current_time);
    strcpy(buffer, ctime(&current_time));
    sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr *)&client_addr, len);
    }
  close(sockfd);
  return 0;
}
```

Timeclient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
   int sockfd;
```

```
    char buffer[BUFFER_SIZE] = "TIME";
    struct sockaddr_in server_addr;
    socklen_t len;
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    sendto(sockfd, buffer, strlen(buffer), 0,(struct sockaddr *)&server_addr,sizeof(server_addr));
    len = sizeof(server_addr);
    recvfrom(sockfd, buffer, BUFFER_SIZE, 0,(struct sockaddr *)&server_addr, &len);
    printf("Server Time: %s\n", buffer);
    close(sockfd);
    return 0;
}
```

Output

```
student@cse-hwl-030:~/surya/socket pgm/c$ cc timeserver.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
UDP Time Server Running…

student@cse-hwl-030:~/surya/socket pgm/c$ cc timeclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Server Time: Thu Jun 18 11:18:36 2026
```

Exp.9. Develop a concurrent file server

Aim:-a concurrent file server that will provide the file requested by the client if it exists. If not, the server sends an appropriate message to the client. The server should also send its process ID (PID) to clients for displaying along with the file contents or with the message.

Algorithm

Server Side

1. Start.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set IP address to `INADDR_ANY`.
   - Set port number to `8080`.
4. Bind the socket to the server address using `bind()`.
5. Put the socket into listening mode using `listen()`.
6. Display **"File Server Running..."**.
7. Enter an infinite loop.
8. Accept a client connection using `accept()`.
9. Create a child process using `fork()`.
10. In the child process:
    - Close the listening socket.
    - Receive the filename from the client using `recv()`.
    - Open the requested file using `fopen()`.
    - Send the child process ID (PID) to the client.
11. Check whether the file exists:
    - If the file does not exist:
      - Send **"Requested file does not exist."** to the client.
    - Otherwise:
      - Read the file line by line using `fgets()`.
      - Send each line to the client using `send()`.
      - Close the file.
12. Close the client socket.
13. Terminate the child process.
14. In the parent process:
    - Close the connected client socket.
    - Continue waiting for new clients.
15. Repeat Steps 8–14 for every client request.
16. Stop when the server is terminated.

Client Side

1. Start.
2. Create a TCP socket using `socket()`.
3. Initialize the server address structure:
   - Set address family to `AF_INET`.
   - Set port number to `8080`.

- Set server IP address to `127.0.0.1`.

4. Connect to the server using `connect()`.
5. Prompt the user to enter a filename.
6. Read the filename from the keyboard.
7. Send the filename to the server using `send()`.
8. Display **"Response from Server:"**.
9. Continuously receive data from the server using `recv()`.
10. Print the received data on the screen.
11. Continue until the server closes the connection.
12. Close the socket.
13. Stop.

Program

Fileserver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main()
{
 int server_fd, client_socket;
 struct sockaddr_in server_addr, client_addr;
 socklen_t addr_len;
 char filename[100];
 char buffer[BUFFER_SIZE];
 server_fd = socket(AF_INET, SOCK_STREAM, 0);
 server_addr.sin_family = AF_INET;
 server_addr.sin_addr.s_addr = INADDR_ANY;
 server_addr.sin_port = htons(PORT);
 bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
 listen(server_fd, 5);
 printf("File Server Running...\n");
 while(1)
   {
  addr_len = sizeof(client_addr);
  client_socket = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len);
  if(fork() == 0)
    {
    close(server_fd);
    recv(client_socket, filename, sizeof(filename), 0);
    FILE *fp = fopen(filename, "r");
    sprintf(buffer, "Server PID: %d\n\n", getpid());
    send(client_socket, buffer, strlen(buffer), 0);
```

```
    if(fp == NULL)
      {
      strcpy(buffer, "Requested file does not exist.\n");
      send(client_socket, buffer, strlen(buffer), 0);
      }
      else
        {
        while(fgets(buffer, BUFFER_SIZE, fp) != NULL)
          {
          send(client_socket, buffer, strlen(buffer), 0);
          }
        fclose(fp);
        }
      close(client_socket);
      exit(0);
    }
    close(client_socket);
    }
    return 0;
    }
```

Fileclient.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main()
  {
   int sockfd;
   struct sockaddr_in server_addr;
   char filename[100];
   char buffer[BUFFER_SIZE];
   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   server_addr.sin_family = AF_INET;
   server_addr.sin_port = htons(PORT);
   server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
   connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));
   printf("Enter filename: ");
   scanf("%s", filename);
   send(sockfd, filename, strlen(filename) + 1, 0);
   printf("\nResponse from Server:\n\n");
   while(recv(sockfd, buffer, BUFFER_SIZE - 1, 0) > 0)
     {
     buffer[BUFFER_SIZE - 1] = '\0';
     printf("%s", buffer);
     }
```

```
   close(sockfd);
   return 0;
 }
```

sample.txt

welcome to networks lab

Output

```
sstudent@cse-hwl-030:~/surya/socket pgm/c$ cc fileserver.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
File Server Running...

student@cse-hwl-030:~/surya/socket pgm/c$ cc fileclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Enter filename: test.txt

Response from Server:
Server PID: 7779

Requested file does not exist.
student@cse-hwl-030:~/surya/socket pgm/c$ cc fileclient.c
student@cse-hwl-030:~/surya/socket pgm/c$ ./a.out
Enter filename: sample.txt

Response from Server:

Server PID: 7792

welcome to networks lab
```

Exp. 10. Packet-capturing application using raw sockets.

Aim:- Develop a packet-capturing application using raw sockets.

Algorithm

1. Start.
2. Create a raw socket using:
   - socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL))
   - This allows capturing all Ethernet packets.
3. Check whether the socket is created successfully.
   - If socket creation fails:
     - Display an error message.
     - Terminate the program.
4. Allocate memory for a packet buffer of size 65536 bytes.
5. Display **"Packet Capture Started..."**.
6. Initialize the packet counter to 0.
7. Enter an infinite loop.
8. Receive a packet from the network using recvfrom().
9. Check whether packet reception is successful.
   - If an error occurs:
     - Display an error message.
     - Exit the loop.
10. Extract the IP header from the received packet:
    - Skip the 14-byte Ethernet header.
    - Cast the remaining data to an IP header structure (iphdr).
11. Obtain:
    - Source IP address (saddr)
    - Destination IP address (daddr)
12. Convert the IP addresses into readable format using inet_ntoa().
13. Increment the packet counter.
14. Display:
    - Packet Number
    - Packet Size
    - Source IP Address
    - Destination IP Address
15. Display a separator line.
16. Repeat Steps 8–15 for every incoming packet.
17. When the program terminates:
    - Close the raw socket.
    - Free the allocated memory.
18. Stop.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <linux/if_packet.h>
```

```
#include <linux/if_ether.h>
#include <netinet/ip.h>
#define BUFFER_SIZE 65536
int main() {
  int sock_raw;
  unsigned char *buffer;
  int packet_count = 0;
  sock_raw = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
  if(sock_raw < 0)  {
    perror("Socket Error"); return 1;  }
  buffer = (unsigned char *)malloc(BUFFER_SIZE);
  printf("Packet Capture Started...\n");
  while(1)   {
    int data_size;
    data_size = recvfrom(sock_raw, buffer, BUFFER_SIZE, 0, NULL, NULL);
    if(data_size < 0)  {
      perror("Receive Error"); break; }
    struct iphdr *ip_header = (struct iphdr *)(buffer + 14);
    struct sockaddr_in src, dst; src.sin_addr.s_addr = ip_header->saddr;
    dst.sin_addr.s_addr = ip_header->daddr;
    printf("\nPacket No: %d\n", ++packet_count);
    printf("Packet Size : %d bytes\n", data_size);
    printf("Source IP : %s\n", inet_ntoa(src.sin_addr));
    printf("Destination IP : %s\n", inet_ntoa(dst.sin_addr));
    printf("--------------------------\n"); }
  close(sock_raw);
  free(buffer);
  return 0; }
```

Output

```
cse@cse-hwl-030:~$ cc packet.c
cse@cse-hwl-030:~$ sudo ./a.out
[sudo] password for cse:
Packet Capture Started...
Packet No: 1
Packet Size : 87 bytes
Source IP : 10.100.74.66
Destination IP : 224.0.0.251
--------------------------
Packet No: 2
Packet Size : 87 bytes
Source IP : 10.100.74.66
Destination IP : 224.0.0.251
--------------------------
Packet No: 3
Packet Size : 60 bytes
Source IP : 88.60.192.168
Destination IP : 0.239.0.0
--------------------------
```

Exp. 11. Familiarizing router commands

Aim:- To familiarize with basic Cisco router commands,router modes,interface configuration, routing information, and configuration management.

(a) Knowing the current mode (user or privileged), switching to privileged mode

(b) Switching to configuration mode

(c) Obtaining router information such as type, OS, memory stats, interface details etc.

(d) Viewing the status of any routing protocols currently configured

(e) Showing the routing table

(f) Saving the running configuration

(g) Viewing the command history

(h) Viewing the router clock

(i) Viewing the list of hosts

(j) Displaying the statistics for all the interfaces (Both detailed and brief views)

(k) Knowing the controller type (DTE or DCE)

(l) Configuring serial and ethernet interfaces - enabling the interface, setting IP address, mask, and clock rate

Algorithm / Procedure

1. Access the router through the console.
2. Identify the current router mode (User EXEC or Privileged EXEC).
3. Enter Privileged EXEC mode using the `enable` command.
4. Enter Global Configuration mode using `configure terminal`.
5. Display router hardware and software information.
6. Check the status of configured routing protocols.
7. View the routing table.
8. Save the current running configuration.
9. Display previously executed commands.
10. View the router clock and host table.
11. Display detailed and brief interface statistics.
12. Check whether the serial interface is DTE or DCE.
13. Configure Ethernet and Serial interfaces with IP addresses and clock rate.
14. Verify the configuration using appropriate show commands.

(a) Knowing the current mode (user or privileged), switching to privileged mode

Check the current mode
Router; → User EXEC Mode
Router# → Privileged EXEC Mode
Switch to Privileged Mode
Syntax:
Router>enable
Router#
To return to User Mode:
Syntax:
Router# disable
Router>

(b) Switching to configuration mode

Router# configure terminal
Router(config)#

(c) Obtaining router information such as type, OS, memory stats, interface details etc.

Router# show version

Cisco IOS Software, Version 15.1
Router Model: 2811
Processor Board ID FTX123456
256 MB RAM
System image file: flash:c2800nm.bin

Router# show interfaces

Observation

Displays:

- Interface names
- MAC addresses
- MTU size
- Bandwidth
- Packet statistics

(d) Viewing the status of any routing protocols currently configured

Router# show ip protocols

Output

Routing Protocol is "rip"
Sending updates every 30 seconds
Routing for Networks:
192.168.1.0

10.0.0.0
(e) Showing the routing table

Router# show ip route
C 192.168.1.0/24 is directly connected, FastEthernet0/0
C 10.0.0.0/8 is directly connected, Serial0/0/0
(f) Saving the running configuration

Router# copy running-config startup-config

Output

Destination filename [startup-config]?
Building configuration...
[OK]
(g) Viewing the command history

Router# show history

Output

enable

configure terminal

show version

show ip route

show history

(h) Viewing the router clock

Router# show clock

Output

14:25:30.123 UTC Mon Mar 10 2025

(i) Viewing the list of hosts

Router# show hosts

Output

Default domain is not set

Name Servers are 8.8.8.8

Host	Address
Server1	192.168.1.100
PC1	192.168.1.101

(j) Displaying the statistics for all the interfaces (Both detailed and brief views)

Router# show interfaces

Output

FastEthernet0/0 is up, line protocol is up

Hardware is Fast Ethernet

MTU 1500 bytes

BW 100000 Kbit

5 minute input rate 2000 bits/sec

5 minute output rate 1000 bits/sec

(k) Knowing the controller type (DTE or DCE)

Router# show controllers serial 0/0/0

Output

Interface Serial0/0/0

Hardware is HD64570

DCE V.35 cable

Observation

DCE device provides clock signal.

DTE device receives clock signal

(l) Configuring serial and ethernet interfaces - enabling the interface, setting IP address, mask, and clock rate

Ethernet Configuration

Commands

Router# configure terminal
Router(config)# interface fastethernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown

Verification

Router# show ip interface brief

Output

FastEthernet0/0 192.168.1.1 YES manual up up

Serial Configuration

Commands

Router(config)# interface serial0/0/0
Router(config-if)# ip address 10.0.0.1 255.0.0.0
Router(config-if)# clock rate 64000
Router(config-if)# no shutdown

Verification

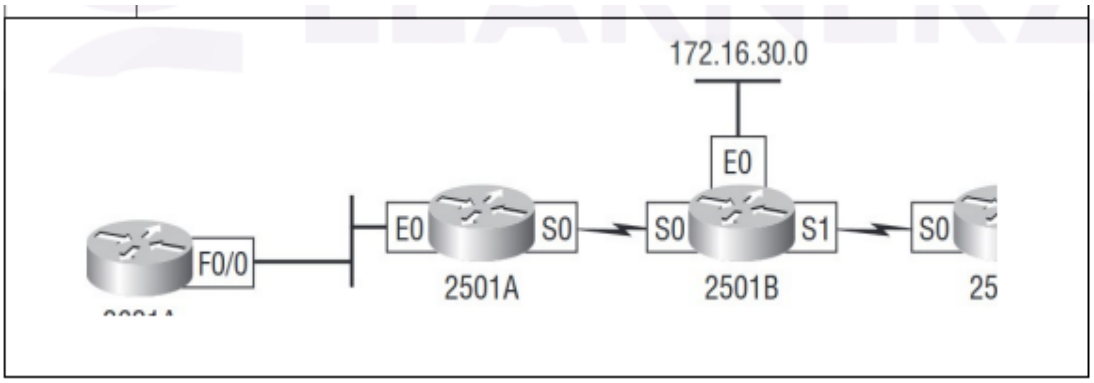
Router# show ip interface brief

Output

Serial0/0/0 10.0.0.1 YES manual up up

Exp. 12. Set up static routing for the network shown in Figure 1. Once the routes are set up, display the routing table and verify the connectivity using ping.

Aim: To configure static routing between routers and verify end-to-end connectivity using routing table inspection and ping commands.



router	Interface	I
2621	F0/0	1
2501A	E0	1
2501A	S0	1
2501B	E0	1
2501B	S0	1
2501B	S1	1

Figure 1: A sample network along with the interface addresses (all interfaces use a /24 mask)

Configuration

Router 2621

Router>enable
Router#configure terminal
Router(config)#
Router(config)# ip route 172.16.20.0 255.255.255.0 172.16.10.2
Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.10.2
Router(config)# ip route 172.16.40.0 255.255.255.0 172.16.10.2

Router 2501A

Router>enable
Router#configure terminal
Router(config)#
Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.20.2
Router(config)# ip route 172.16.40.0 255.255.255.0 172.16.20.2

Router 2501B

Router>enable
Router#configure terminal
Router(config)#
Router(config)# ip route 172.16.10.0 255.255.255.0 172.16.20.1

Router 2501C

Router>enable
Router#configure terminal
Router(config)#
Router(config)# ip route 172.16.10.0 255.255.255.0 172.16.40.1
Router(config)# ip route 172.16.20.0 255.255.255.0 172.16.40.1
Router(config)# ip route 172.16.30.0 255.255.255.0 172.16.40.1

Displaying the Routing Table

Router# show ip route

Sample Output (Router 2621)

Router# show ip route
C 172.16.10.0/24 is directly connected, FastEthernet0/0
S 172.16.20.0/24 [1/0] via 172.16.10.2
S 172.16.30.0/24 [1/0] via 172.16.10.2
S 172.16.40.0/24 [1/0] via 172.16.10.2

Where:

- C = Connected Route
- S = Static Route

Connectivity Verification

Ping from Router 2621 to Router 2501B LAN Interface

Router# ping 172.16.30.1

Expected Output:

Success rate is 100 percent (5/5)

Ping from Router 2621 to Router 2501C

Router# ping 172.16.40.2

Expected Output:

Success rate is 100 percent (5/5)

Ping from Router 2501C to Router 2621

Router# ping 172.16.10.1

Expected Output:

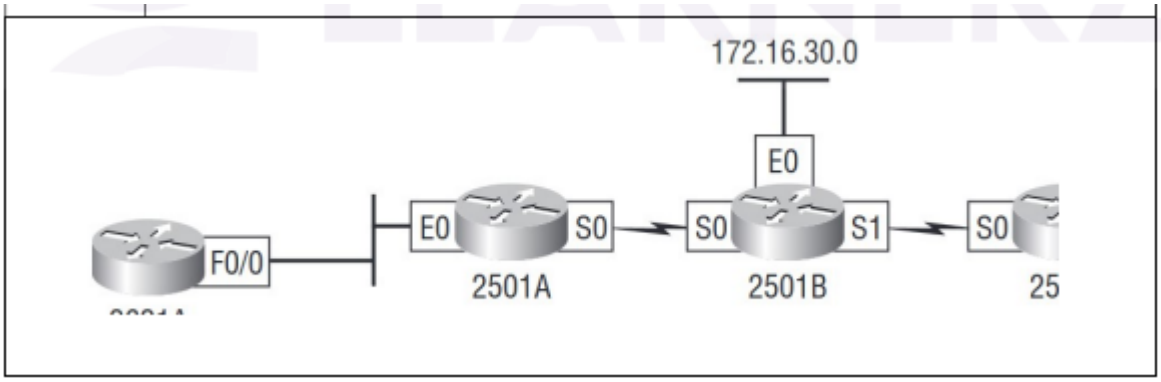
Success rate is 100 percent (5/5)

Result

Static routes were successfully configured on all routers. The routing tables displayed the configured static routes, and connectivity between all networks was verified successfully using the ping command. Hence, static routing was implemented and tested successfully.

Exp. 13. Implement RIPv2 routing for the network shown in Figure 1. Once the routes are set up, display the routing table and verify the connectivity using ping.

Aim: To configure RIPv2 (Routing Information Protocol Version 2) on the routers in the given network topology and verify connectivity using the routing table and ping commands.



router	Interface	I
2621	F0/0	1
2501A	E0	1
2501A	S0	1
2501B	E0	1
2501B	S0	1
2501B	S1	1

Figure 1: A sample network along with the interface addresses (all interfaces use a /24 mask)

Configuration

Router 2621

```
Router>enable
Router#configure terminal
Router(config)#
Router(config)# router rip
Router(config-router)# version 2
Router(config-router)# no auto-summary
Router(config-router)# network 172.16.10.0
```

Router 2501A

Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.10.0

Router(config-router)# network 172.16.20.0

Router 2501B

Router>enable
Router#configure terminal
Router(config)#
Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.20.0

Router(config-router)# network 172.16.30.0

Router(config-router)# network 172.16.40.0

Router 2501C

Router>enable
Router#configure terminal
Router(config)#
Router(config)# router rip

Router(config-router)# version 2

Router(config-router)# no auto-summary

Router(config-router)# network 172.16.40.0

Verify RIP Configuration

Router# show ip protocols

Sample Output

Routing Protocol is "rip"

Sending updates every 30 seconds

Invalid after 180 seconds

Hold down 180 seconds

Flush after 240 seconds

Routing for Networks:

172.16.10.0

172.16.20.0

Display Routing Table

Router# show ip route

Sample Output (Router 2621)

Router# show ip route
C 172.16.10.0/24 is directly connected, FastEthernet0/0

R 172.16.20.0/24 [120/1] via 172.16.10.2

R 172.16.30.0/24 [120/2] via 172.16.10.2

R 172.16.40.0/24 [120/3] via 172.16.10.2

Where:

- C = Directly Connected Route
- R = RIP Learned Route
- 120 = Administrative Distance of RIP

Connectivity Verification

Ping from Router 2621 to Router 2501B LAN

Router# ping 172.16.30.1

Output:
Success rate is 100 percent (5/5)

Ping from Router 2621 to Router 2501C

Router# ping 172.16.40.2

Output:
Success rate is 100 percent (5/5)

Ping from Router 2501C to Router 2621

Router# ping 172.16.10.1

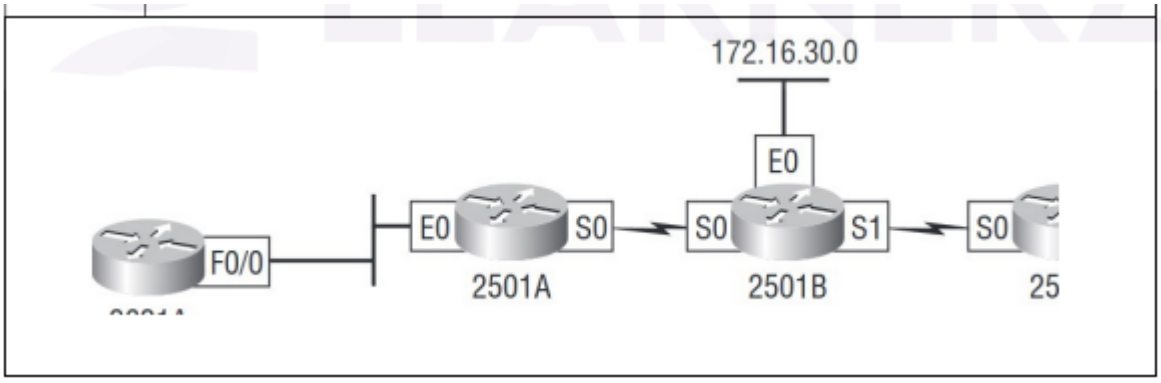
Output:
Success rate is 100 percent (5/5)

Result

RIPv2 routing was successfully configured on all routers. The routing tables contained dynamically learned RIP routes, and connectivity between all networks was verified successfully using the ping command. Hence, the implementation of RIPv2 routing was successful.

Exp. 14. Implement OSPF routing for the network shown in Figure 1. Once the routes are set up,display the routing table and verify the connectivity using ping.

Aim: To configure OSPF (Open Shortest Path First) routing on the routers in the given network topology and verify connectivity using the routing table and ping commands.



router	Interface	I
2621	F0/0	1
2501A	E0	1
2501A	S0	1
2501B	E0	1
2501B	S0	1
2501B	S1	1

Figure 1: A sample network along with the interface addresses (all interfaces use a /24 mask)

Configuration

Router 2621

```
Router>enable
Router#configure terminal
Router(config)#
Router(config)# router ospf 1
Router(config-router)# network 172.16.10.0 0.0.0.255 area 0
```

Router 2501A

```
Router>enable
Router#configure terminal
Router(config)#
Router(config)# router ospf 1
Router(config-router)# network 172.16.10.0 0.0.0.255 area 0
Router(config-router)# network 172.16.20.0 0.0.0.255 area 0
```

Router 2501B

```
Router>enable
Router#configure terminal
Router(config)#
Router(config)# router ospf 1
Router(config-router)# network 172.16.20.0 0.0.0.255 area 0
Router(config-router)# network 172.16.30.0 0.0.0.255 area 0
Router(config-router)# network 172.16.40.0 0.0.0.255 area 0
```

Router 2501C

```
Router>enable
Router#configure terminal
Router(config)#
Router(config)# router ospf 1
Router(config-router)# network 172.16.40.0 0.0.0.255 area 0
```

Verify OSPF Neighbor Relationships

Router# show ip ospf neighbor

Sample Output

```
Neighbor ID     Pri   State           Dead Time   Address
172.16.20.2       1   FULL/ -         00:00:38    172.16.20.2
```

This confirms that OSPF adjacency has been established successfully.

Display OSPF Routing Table

Router# show ip route

Sample Output (Router 2621)

Router# show ip route

```
C 172.16.10.0/24 is directly connected, FastEthernet0/0
O 172.16.20.0/24 [110/65] via 172.16.10.2
O 172.16.30.0/24 [110/129] via 172.16.10.2
O 172.16.40.0/24 [110/193] via 172.16.10.2
```

Where:

- **C** = Connected Route
- **O** = OSPF Learned Route
- **110** = Administrative Distance of OSPF

Verify OSPF Database

Router# show ip ospf database

This command displays all LSAs (Link State Advertisements) exchanged among OSPF routers.

Connectivity Verification

Ping from Router 2621 to Router 2501B LAN

Router# ping 172.16.30.1

Output:

Success rate is 100 percent (5/5)

Ping from Router 2621 to Router 2501C

Router# ping 172.16.40.2

Output:

Success rate is 100 percent (5/5)

Ping from Router 2501C to Router 2621

Router# ping 172.16.10.1

Output:

Success rate is 100 percent (5/5)

Result

OSPF routing was successfully configured on all routers. The routing tables displayed OSPF-learned routes, and connectivity between all networks was verified using the ping command. Hence, the implementation of **OSPF routing** was successful.

Exp. 15. You are the network administrator of your college. A small portion of your campus network is shown in Figure 2. You want to allow only Host_B to communicate with the network 172.16.10.0. Verify your settings by the following checks:

(a) Pinging Host_A from Host_B

(b) Pinging Host_A from Lab_B and Lab_C

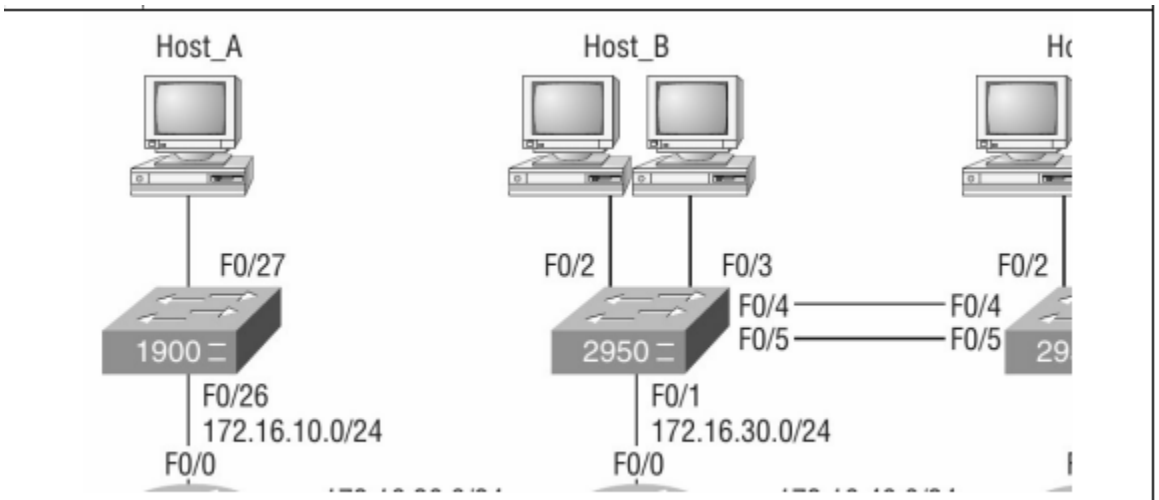


Figure 2: A portion of your college campus network

Aim: To configure an Extended ACL so that only Host_B is allowed to communicate with the network 172.16.10.0/24, while access from Lab_B and Lab_C is denied.

Topology Information

From Figure 2:

- Host_A belongs to network 172.16.10.0/24
- Host_B belongs to network 172.16.30.0/24
- Lab_B and Lab_C are also connected to the 172.16.30.0/24 network
- Requirement: Allow only Host_B to access Host_A's network (172.16.10.0/24)

Assume:

Device	IP Address
Host_A	172.16.10.10
Host_B	172.16.30.10
Lab_B	172.16.30.11
Lab_C	172.16.30.12

ACL Configuration

Configure the Extended ACL on the router interface closest to the source network

(172.16.30.0/24).

Router(config)# access-list 101 permit ip host 172.16.30.10 172.16.10.0 0.0.0.255

Router(config)# access-list 101 deny ip 172.16.30.0 0.0.0.255 172.16.10.0 0.0.0.255

Router(config)# access-list 101 permit ip any any

Apply the ACL to the interface connected to the 172.16.30.0 network:

Router(config)# interface f0/0

Router(config-if)# ip access-group 101 in

Verification

Display ACL

Router# show access-lists

Sample Output

Extended IP access list 101

10 permit ip host 172.16.30.10 172.16.10.0 0.0.0.255

20 deny ip 172.16.30.0 0.0.0.255 172.16.10.0 0.0.0.255

30 permit ip any any

Connectivity Tests

(a) Ping Host_A from Host_B

Host_B> ping 172.16.10.10

Expected Result

Reply from 172.16.10.10

Reply from 172.16.10.10

Reply from 172.16.10.10

Reply from 172.16.10.10

Success rate is 100%

Observation: Communication is successful because Host_B is explicitly permitted by the ACL.

(b) Ping Host_A from Lab_B

Lab_B> ping 172.16.10.10

Expected Result

Request timed out.

Request timed out.

Request timed out.

Request timed out.

Success rate is 0%

Observation: Communication is blocked by the ACL.

(c) Ping Host_A from Lab_C

Lab_C> ping 172.16.10.10

Expected Result
Request timed out.

Request timed out.

Request timed out.

Request timed out.

Success rate is 0%

Observation: Communication is blocked by the ACL.
Result

An Extended Access Control List (ACL) was successfully configured to allow only Host_B to communicate with the 172.16.10.0/24 network. Connectivity tests confirmed that Host_B could successfully ping Host_A, while Lab_B and Lab_C were denied access. Thus, the ACL configuration was verified successfully.

Exp.16. You are the network administrator of your college. The college is assigned a network address 140.80.0.0. There are 20 subnets in your college network. The Central Computing Facility (CCF) resides in the 4th subnet. The department of CSE is organizing an inter-department hackathon for which the registration closed yesterday. The registration was through the hackathon website hosted on a server which is assigned the 7th address in the 16th subnet. As the network administrator, your job now is to block students from accessing the hackathon website from CCF. [The server provides other services than the website hosting as well. Make sure you block only the website access. Other services should not be denied.].

Aim:

1. Determine the subnet mask for 20 subnets.
2. Find the network addresses of the 4th subnet (CCF) and 16th subnet (server).
3. Create an extended ACL that blocks only web access (HTTP) from the CCF subnet to the server while allowing all other services.

Step 1: Subnetting

Given network: 140.80.0.0

140.80.0.0 is a Class B network (default mask /16).

Need at least 20 subnets:

2n≥202^n \ge 202n≥20 25=322^5 = 3225=32

Borrow 5 bits from the host portion.

New subnet mask:

/16+5=/21/16 + 5 = /21/16+5=/21

Subnet Mask:

255.255.248.0255.255.248.0255.255.248.0

Block size in the 3rd octet:

256−248=8256 - 248 = 8256−248=8

Step 2: Identify the Required Subnets

Subnets are:

Subnet No.	Network Address

1	140.80.0.0/21
2	140.80.8.0/21
3	140.80.16.0/21
4	140.80.24.0/21
5	140.80.32.0/21
...	...
16	140.80.120.0/21

CCF (4th subnet)

Network Address:

140.80.24.0/21

Wildcard mask:

255.255.248.0⇒0.0.7.255255.255.248.0 \Rightarrow 0.0.7.255255.255.248.0⇒0.0.7.255

Server (7th address in 16th subnet)

16th subnet:

140.80.120.0/21

Host addresses start from:

- 140.80.120.1 → 1st host
- 140.80.120.2 → 2nd host
- ...
- 140.80.120.7 → 7th host

Server IP:

140.80.120.7

Step 3: Configure Extended ACL

Block only HTTP (TCP port 80) access from CCF to the server.

access-list 101 deny tcp 140.80.24.0 0.0.7.255 host 140.80.120.7 eq 80

access-list 101 permit ip any any

Apply the ACL on the router interface connected to the CCF subnet (inbound):

interface GigabitEthernet0/0

ip access-group 101 in

Verification

HTTP Access

http://140.80.120.7

Result:

Access Denied

Other Services

Ping:

ping 140.80.120.7

Result:

Reply from 140.80.120.7

FTP/SSH (if enabled):

ftp 140.80.120.7

ssh 140.80.120.7

Result:

Connection Successful

Exp.17. Figure 3 shows an IPv6-based network. Interconnect the different subnets using RIPng.

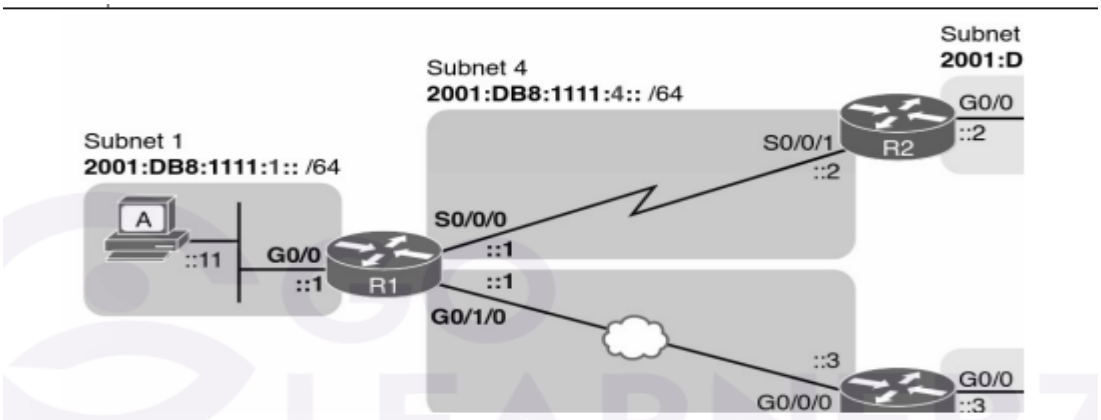


Figure 3: An IPv6 network

IPv6 Addressing Scheme

From Figure 3:

Device	Interface	IPv6 Address
PC-A	NIC	2001:DB8:1111:1::11/64
R1	G0/0	2001:DB8:1111:1::1/64
R1	S0/0/0	2001:DB8:1111:4::1/64
R2	S0/0/1	2001:DB8:1111:4::2/64
R2	G0/0	2001:DB8:1111:2::2/64
R1	G0/1/0	2001:DB8:1111:3::1/64
R3	G0/0/0	2001:DB8:1111:3::3/64
R3	G0/0	2001:DB8:1111:5::3/64

Subnets

- Subnet 1 → 2001:DB8:1111:1::/64
- Subnet 2 → 2001:DB8:1111:2::/64
- Subnet 3 → 2001:DB8:1111:3::/64
- Subnet 4 → 2001:DB8:1111:4::/64
- Subnet 5 → 2001:DB8:1111:5::/64

Router Configuration

R1 Configuration

```
enable
configure terminal

ipv6 unicast-routing

interface g0/0
ipv6 address 2001:DB8:1111:1::1/64
ipv6 rip RIPNG enable
no shutdown

interface s0/0/0
ipv6 address 2001:DB8:1111:4::1/64
ipv6 rip RIPNG enable
no shutdown

interface g0/1/0
ipv6 address 2001:DB8:1111:3::1/64
ipv6 rip RIPNG enable
no shutdown

end
```

R2 Configuration

```
enable
configure terminal
ipv6 unicast-routing

interface s0/0/1
ipv6 address 2001:DB8:1111:4::2/64
ipv6 rip RIPNG enable
no shutdown

interface g0/0
ipv6 address 2001:DB8:1111:2::2/64
ipv6 rip RIPNG enable
no shutdown
end
```

R3 Configuration

```
enable
configure terminal
ipv6 unicast-routing
interface g0/0/0
ipv6 address 2001:DB8:1111:3::3/64
```

ipv6 rip RIPNG enable
no shutdown

interface g0/0
ipv6 address 2001:DB8:1111:5::3/64
ipv6 rip RIPNG enable
no shutdown
end

PC Configuration

PC-A

IPv6 Address : 2001:DB8:1111:1::11/64
Default Gateway : 2001:DB8:1111:1::1

PC-B (Subnet 2)

IPv6 Address : 2001:DB8:1111:2::10/64
Default Gateway : 2001:DB8:1111:2::2

PC-C (Subnet 5)

IPv6 Address : 2001:DB8:1111:5::10/64
Default Gateway : 2001:DB8:1111:5::3

Verification

Display RIPng Information

show ipv6 protocols

Sample Output:

RIP process "RIPNG"
Interfaces:
  GigabitEthernet0/0
  Serial0/0/0
  GigabitEthernet0/1/0

Display Routing Table

show ipv6 route

Sample Output on R1

R1# show ipv6 route

C   2001:DB8:1111:1::/64
C   2001:DB8:1111:3::/64
C   2001:DB8:1111:4::/64
R   2001:DB8:1111:2::/64
    via FE80::2, Serial0/0/0
R   2001:DB8:1111:5::/64
    via FE80::3, GigabitEthernet0/1/0

Sample Output on R2

R2# show ipv6 route

C 2001:DB8:1111:2::/64
C 2001:DB8:1111:4::/64
R 2001:DB8:1111:1::/64
   via FE80::1
R 2001:DB8:1111:3::/64
   via FE80::1
R 2001:DB8:1111:5::/64
   via FE80::1

Connectivity Test

Ping from PC-A to PC-B

ping 2001:DB8:1111:2::10

Output:

Reply from 2001:DB8:1111:2::10
Reply from 2001:DB8:1111:2::10
Reply from 2001:DB8:1111:2::10
Reply from 2001:DB8:1111:2::10

Success rate = 100%

Ping from PC-A to PC-C

ping 2001:DB8:1111:5::10

Output:

Reply from 2001:DB8:1111:5::10
Reply from 2001:DB8:1111:5::10
Reply from 2001:DB8:1111:5::10
Reply from 2001:DB8:1111:5::10
Success rate = 100%

Result

The IPv6 addresses were configured on all router interfaces and hosts. RIPng was enabled on all participating interfaces, allowing dynamic exchange of IPv6 routing information. The routing tables were updated successfully, and connectivity between all IPv6 subnets was verified using ping. Hence, the different IPv6 subnets were successfully interconnected using RIPng.